

Unit 1

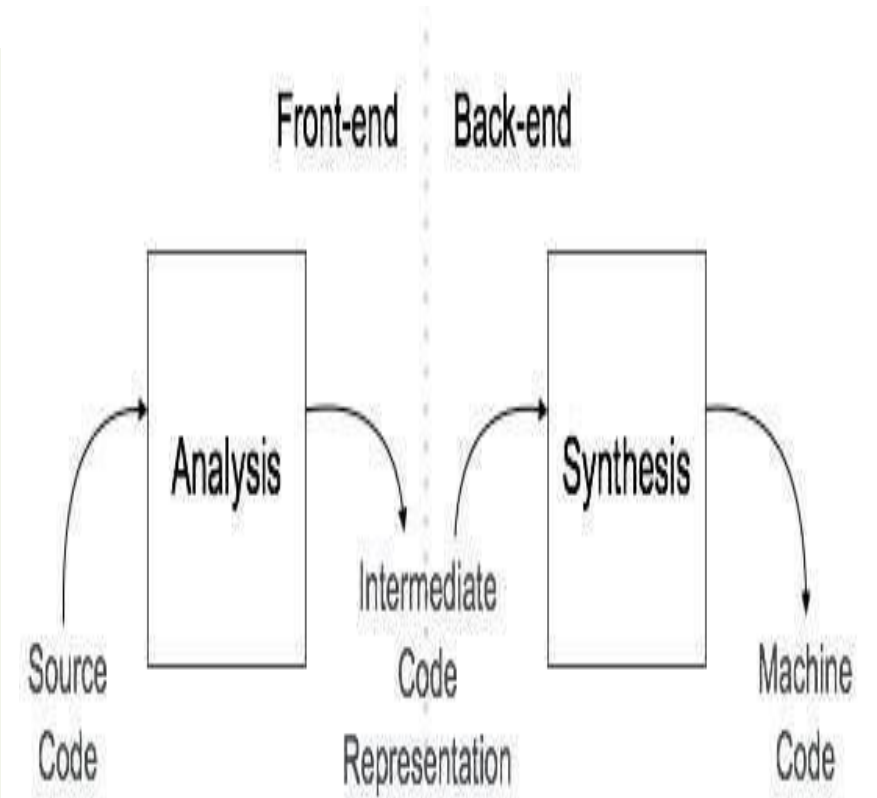
1.1 Compiler Structure

Analysis and Synthesis
Model

Phases of Compilation

1.2 Basic Concepts

- Interpreter
- One-Pass Compiler
- Preprocessor
- Macros
- Symbol Table
- Error Handler

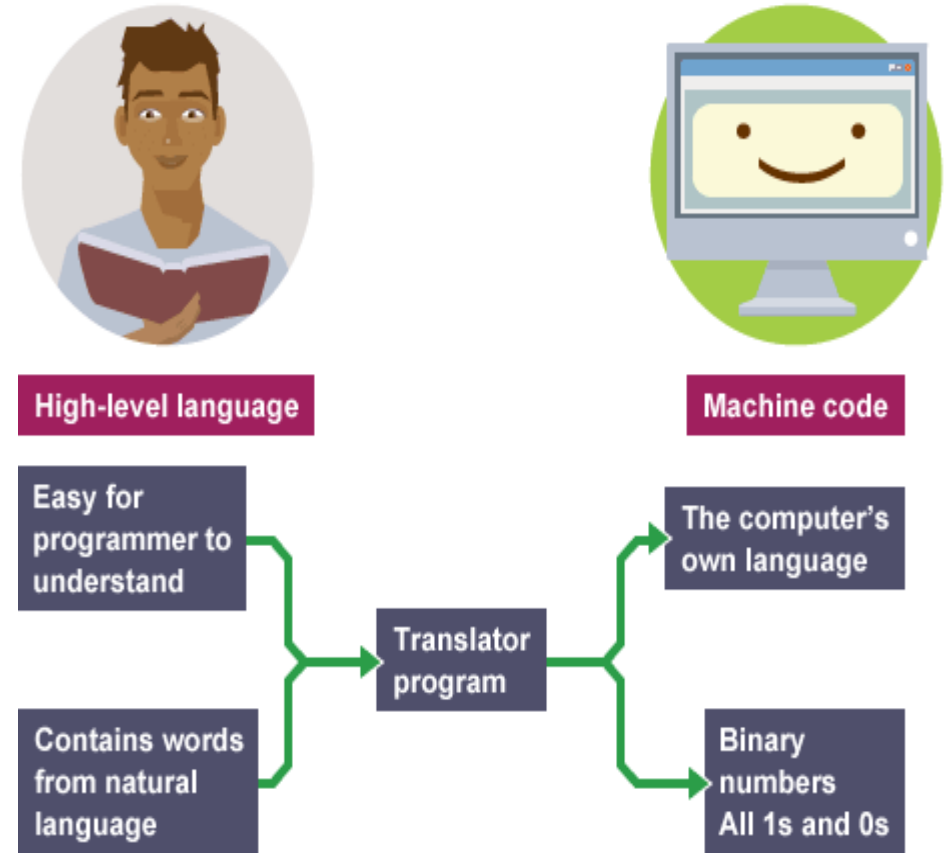


Language Processing

- Language processing is the process of converting a program written in a programming language into machine language so that a computer can understand and execute it.
- A computer can understand only machine language (binary code).
- Therefore, programs written in high-level languages must be translated into machine code before they can be executed.
- It acts like a bridge between human-written code and computer hardware.

- **Major High-Level Languages**

- C
- C++
- Java
- Python



Language Processing Tools

- **Compiler**

- Translates the entire source program into machine code before execution.
- Produces an executable file.
- Example: C, C++

- **Interpreter**

- Translates and executes the program line by line.
- Does not create a separate executable file.
- Example: Python

- **Assembler**

- Converts assembly language into machine language.

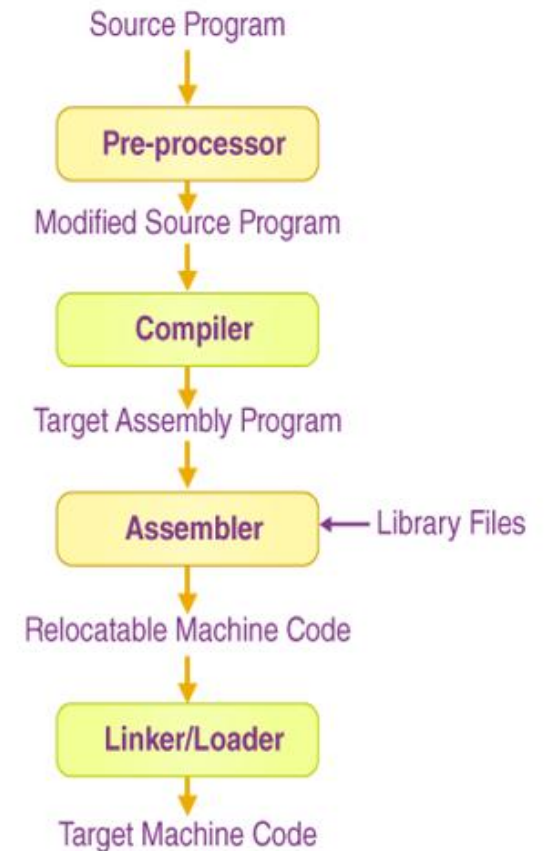
- **Linker**

- Combines object files and required libraries into a single executable program.

- **Loader**

- Loads the executable program into the computer's memory and prepares it for execution.

Steps for Language Processing System



Compiler Introduction

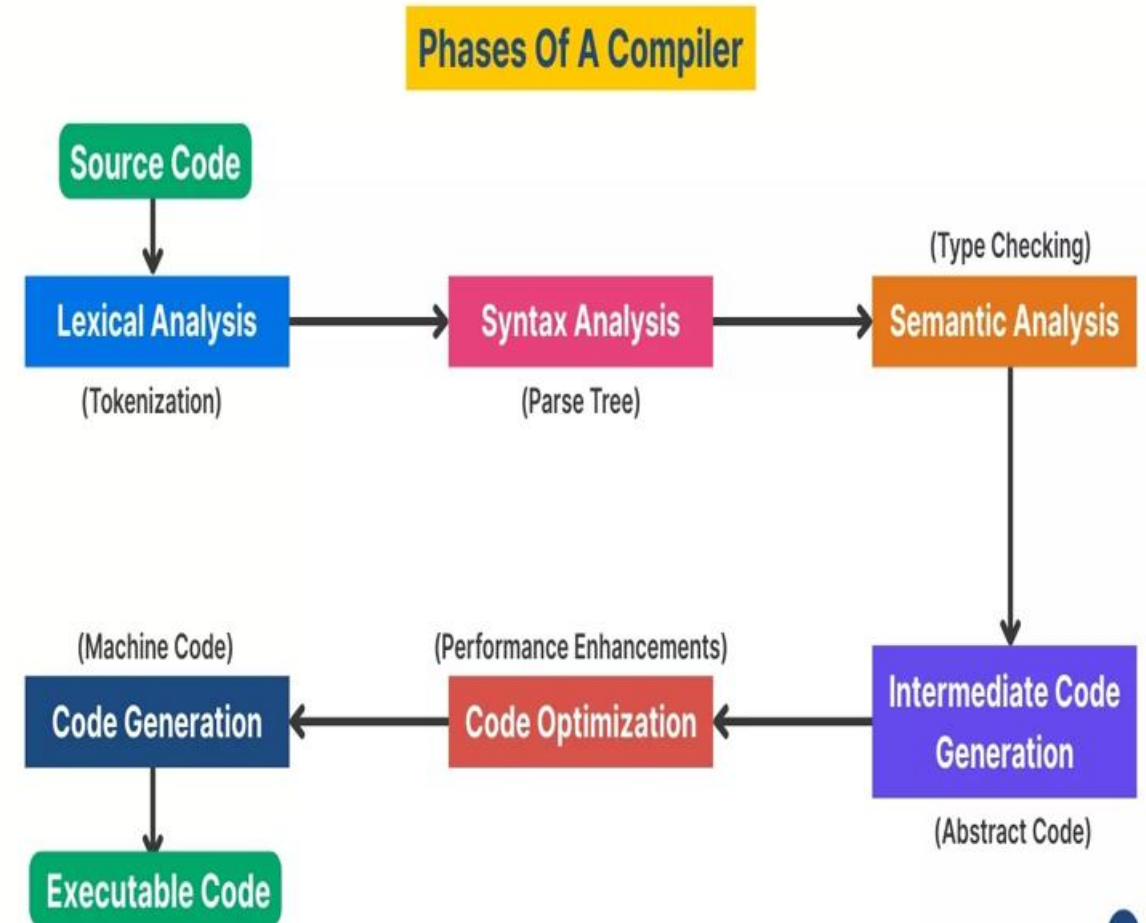
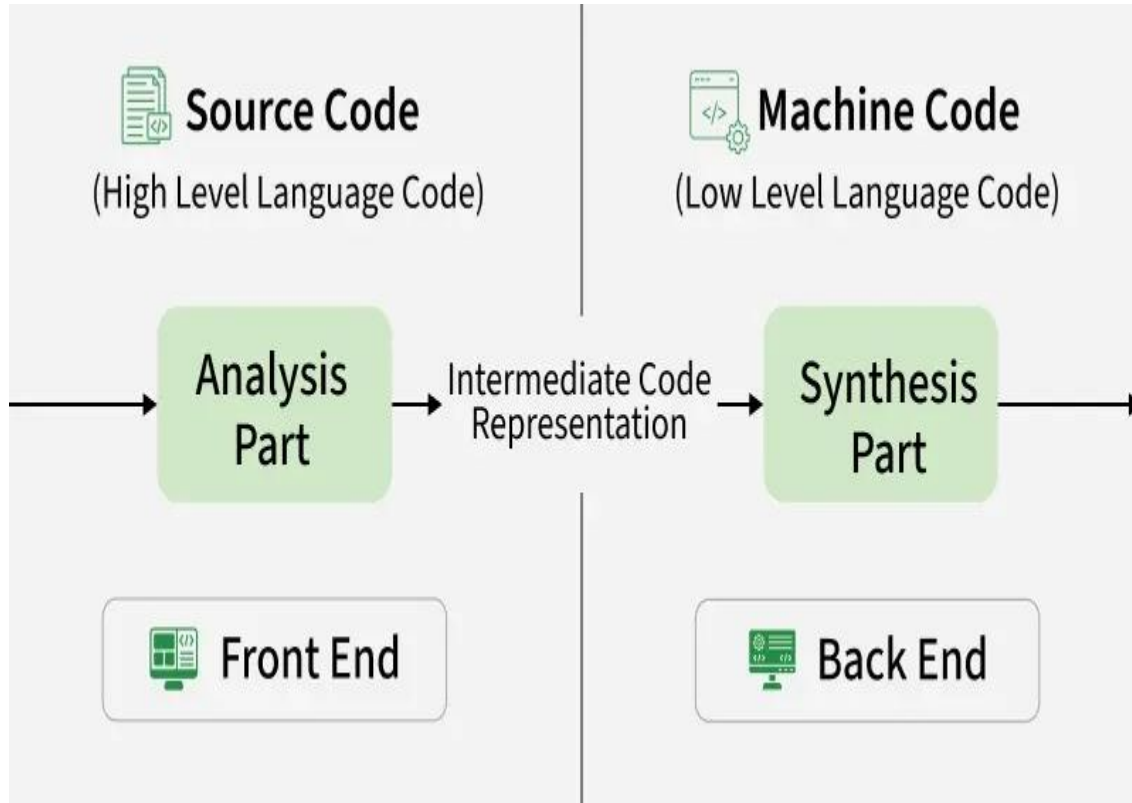
- A compiler is a software program that translates source code written in a high-level language into machine language.
- It reads the entire source code, checks for errors, and then creates an executable (machine code) file.
- Many programming languages, such as **C**, **C++**, and **Java**, use a compiler to translate source code into machine code or another executable form.

Functions of a Compiler

- **Translation:** Converts source code into machine code.
- **Error Detection:** Finds and reports errors in the source code.
- **Optimization:** Improves the code to make it run faster and use fewer resources.
- **Code Generation:** Produces the final machine code that the computer executes.



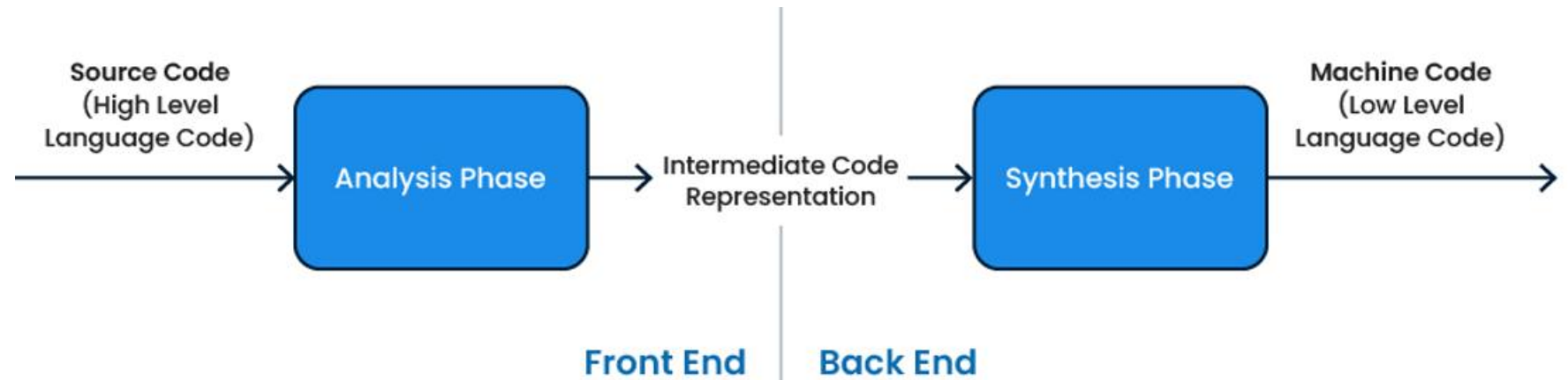
Compiler Structure Overview



Analysis and Synthesis Model

The **Analysis and Synthesis Model** divides the work of a compiler into two main parts:

- **Analysis Phase (Front End)**
 - Examines source program
- **Synthesis Phase (Back End)**
 - Generates target code



1. Analysis Phase (Front End)

- The Analysis Phase, also called the Front End.
- It is the first stage of a compiler.
- It reads the source program, checks it for errors, and understands its structure and meaning.
- After analyzing the program, it creates an Intermediate Representation (IR), which is used by the next phase of the compiler.

Main tasks of the Analysis Phase:

- Lexical Analysis: Reads the source code and groups characters into tokens (such as keywords, identifiers, operators, and constants). It also removes whitespace and comments.
- Syntax Analysis (Parsing): Checks whether the tokens follow the grammar rules of the programming language and creates a parse tree or syntax tree.
- Semantic Analysis: Checks the meaning of the program, such as variable declarations, data types, and type compatibility. It also uses the Symbol Table to verify the correctness of the program.
- Intermediate Code Generation: Converts the analyzed program into an Intermediate Representation (IR), which is easier to optimize and translate into machine code.

2. Synthesis Phase (Back End)

- The Synthesis Phase, also called the Back End.
- It is the second stage of a compiler.
- It takes the Intermediate Representation (IR) created by the Analysis Phase and converts it into the final machine code that the computer can execute.

Main tasks of the Synthesis Phase:

- **Intermediate Code Processing:** Uses the machine-independent Intermediate Representation (IR) produced by the front end as the input for the back end.
- **Code Optimization:** Improves the intermediate code by removing unnecessary operations and making it more efficient. This helps the program run faster and use less memory.
- **Code Generation:** Converts the optimized intermediate code into assembly code or machine code. It also selects the appropriate machine instructions and assigns variables to processor registers.
- The final output of the synthesis phase is the target machine code, which can be executed by the computer.

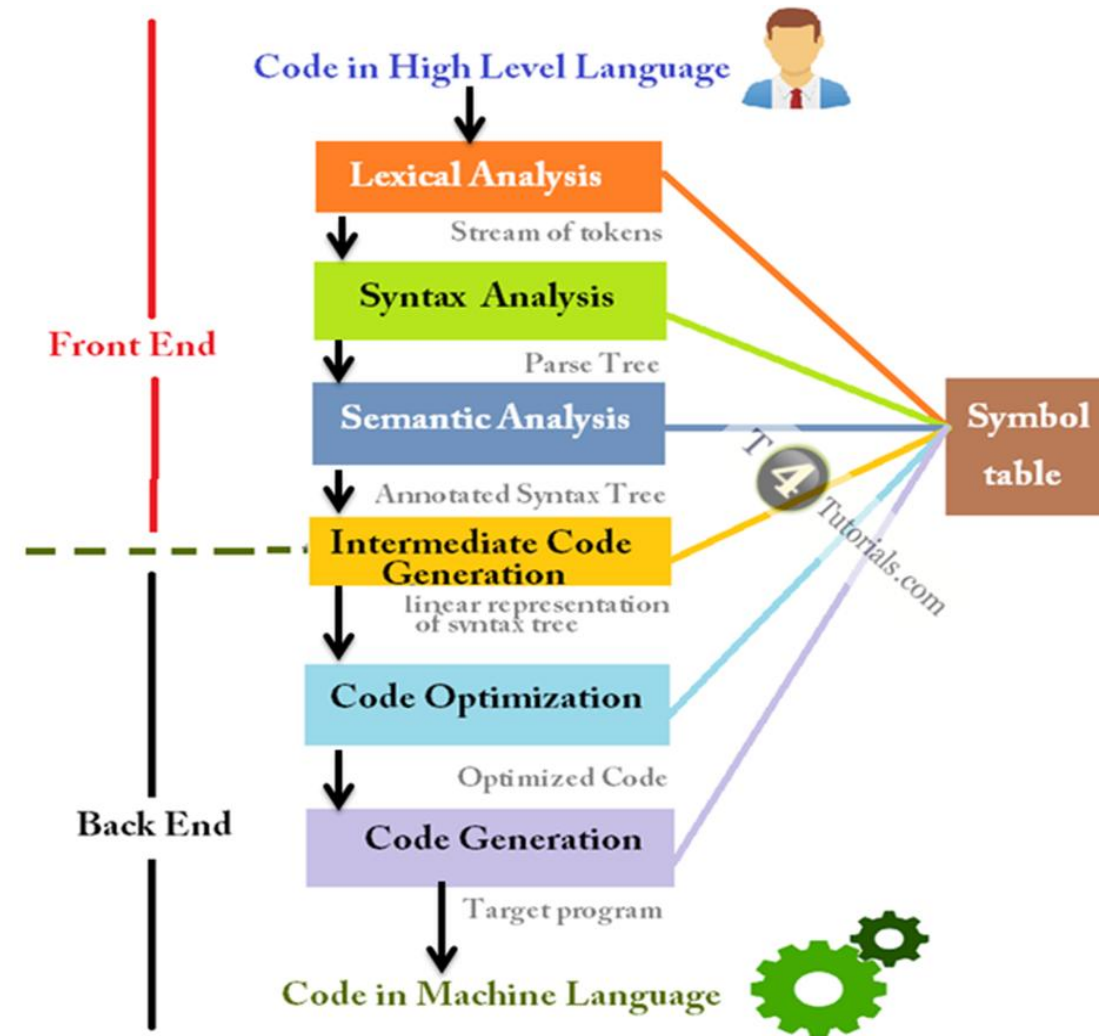
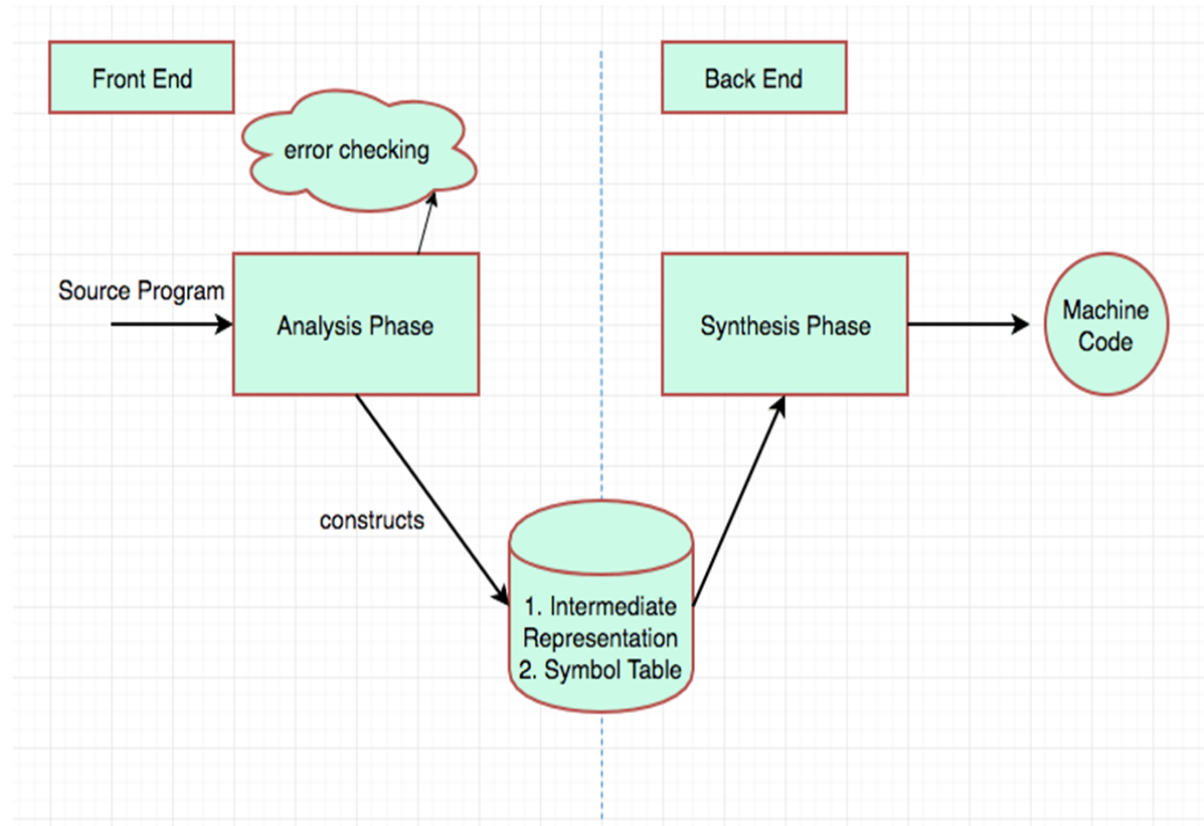


Fig: Analysis and Synthesis Phase of Compiler

Compiler Phases

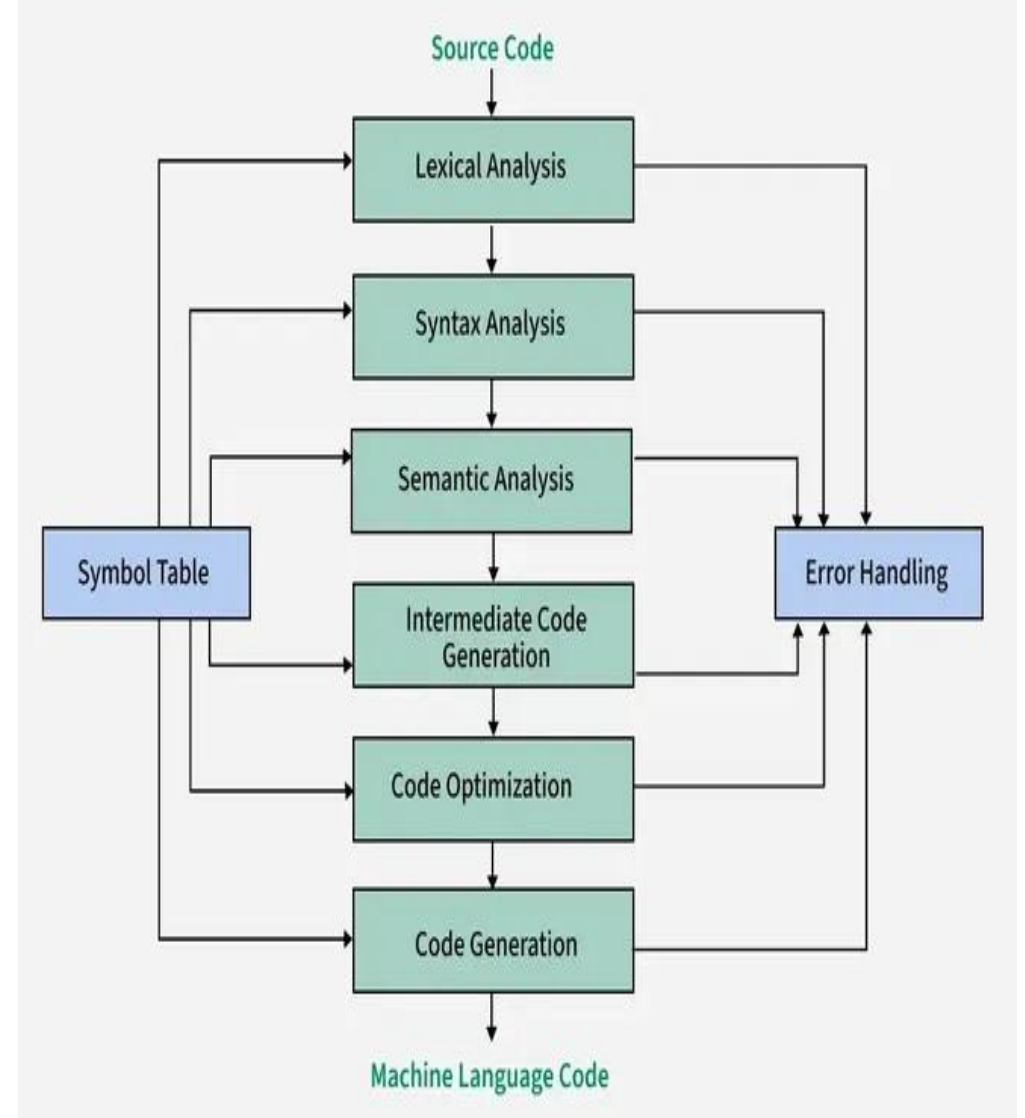
The **compilation process** is divided into six main phases.

Each phase performs a specific task to convert a high-level language program into machine code

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis
4. Intermediate Code Generation
5. Code Optimization
6. Code Generation

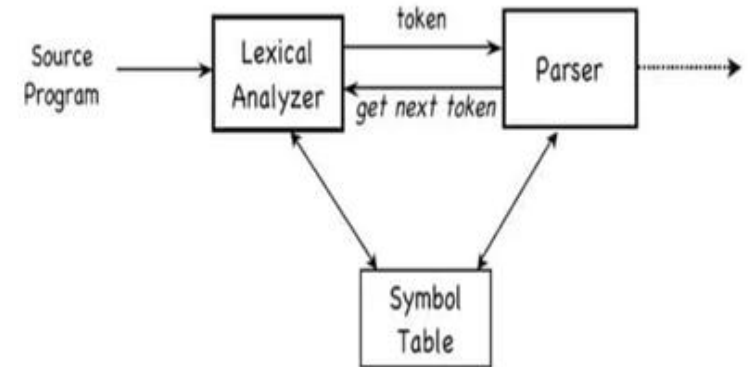
Supporting Components:

- Symbol Table
- Error Handler



Lexical Analysis (Scanner)

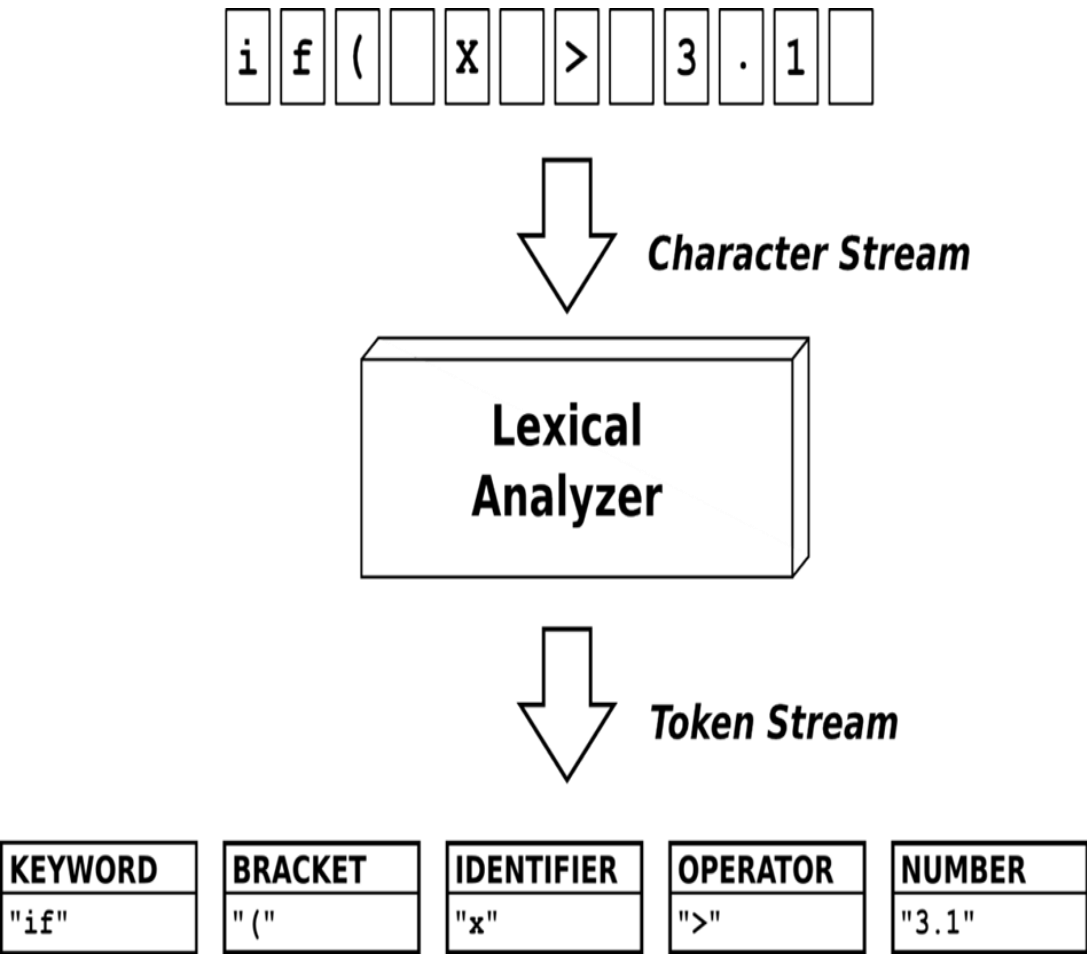
- Lexical Analysis is the first phase of a compiler and is also known as a Scanner.
- It reads the source code as a sequence of characters and converts them into meaningful units called tokens.
- During this phase, the compiler:
 - Groups characters into tokens such as keywords, identifiers, operators, and constants.
 - Removes unnecessary elements like: Whitespace (spaces, tabs, and new lines) Comments
- The main purpose of lexical analysis is to convert the character stream of the source code into a token stream that can be understood and processed by the syntax analyzer.



Example:

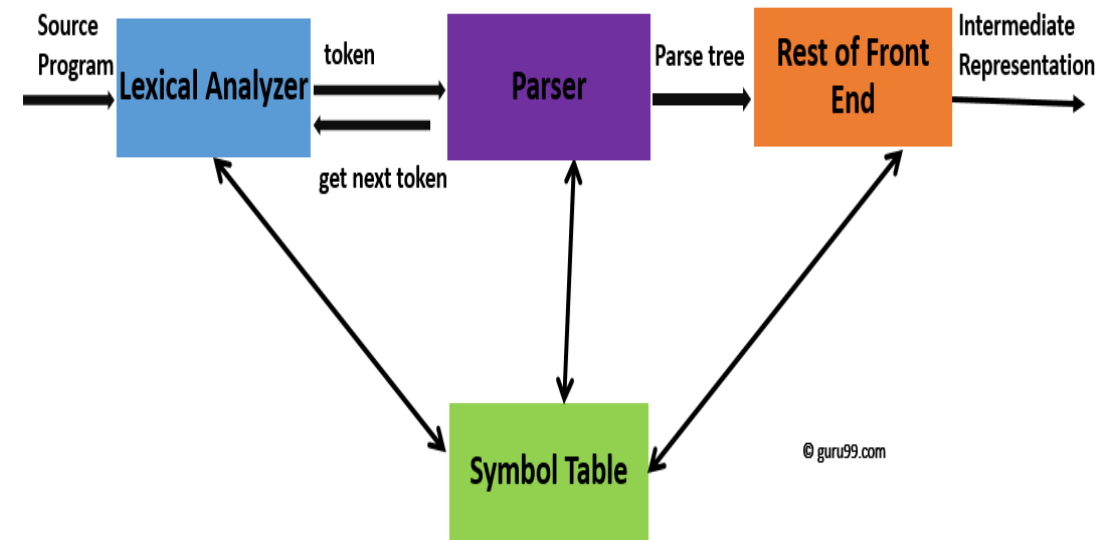
```
int x = 10;
```

Lexeme	Token
int	KEYWORD
x	IDENTIFIER
=	ASSIGN_OP
10	NUMBER
;	SEMICOLON



Syntax Analysis (Parsing)

- Syntax Analysis is the second phase of a compiler and is also known as Parsing.
- It takes the token stream generated by the Lexical Analyzer as input and checks whether the tokens follow the grammar rules of the programming language.
- During this phase, the compiler:
 - Checks the correct arrangement of tokens in the program.
 - Detects syntax errors such as:
 - Missing semicolons
 - Unmatched parentheses
 - Incorrect statement structures
- Creates a Parse Tree that shows the hierarchical structure of the program.
- The main purpose of syntax analysis is to verify the grammatical correctness of the source code and prepare it for the next phase of compilation.



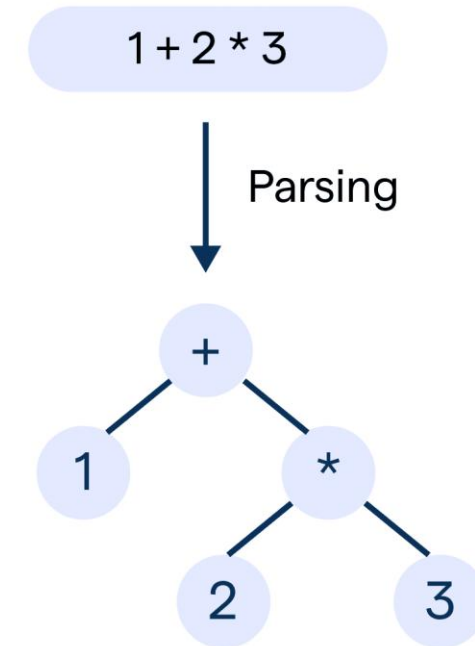
Example:

Int x = 10;

KEYWORD(int)
IDENTIFIER(x)
ASSIGN_OP(=)
NUMBER(10)
SEMICOLON(;

The parser checks whether this sequence tokens follows the grammar rule or not.

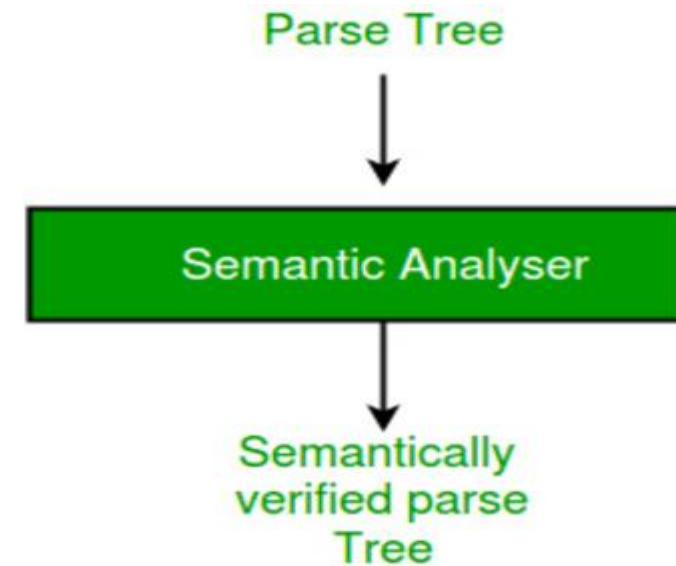
Syntax Analysis



A parse tree is a tree representation showing how a statement conforms to the grammar rules of a programming language.

Semantic Analysis

- Semantic Analysis is the third phase of a compiler.
- It checks the meaning of the program and ensures that the code is logically correct according to the rules of the programming language.
- This phase uses the parse tree created during Syntax Analysis and verifies the semantic correctness of the program.
- It also creates an Annotated Syntax Tree that contains additional information such as data types, variable scope, and symbol table references.
- During Semantic Analysis, the compiler detects errors such as:
 - Type mismatches
 - Use of undeclared variables
 - Invalid operations
 - Incorrect function calls
 - Scope violations
- The main purpose of semantic analysis is to check the meaning and logical correctness of the source code before generating intermediate code.



Intermediate Code Generation

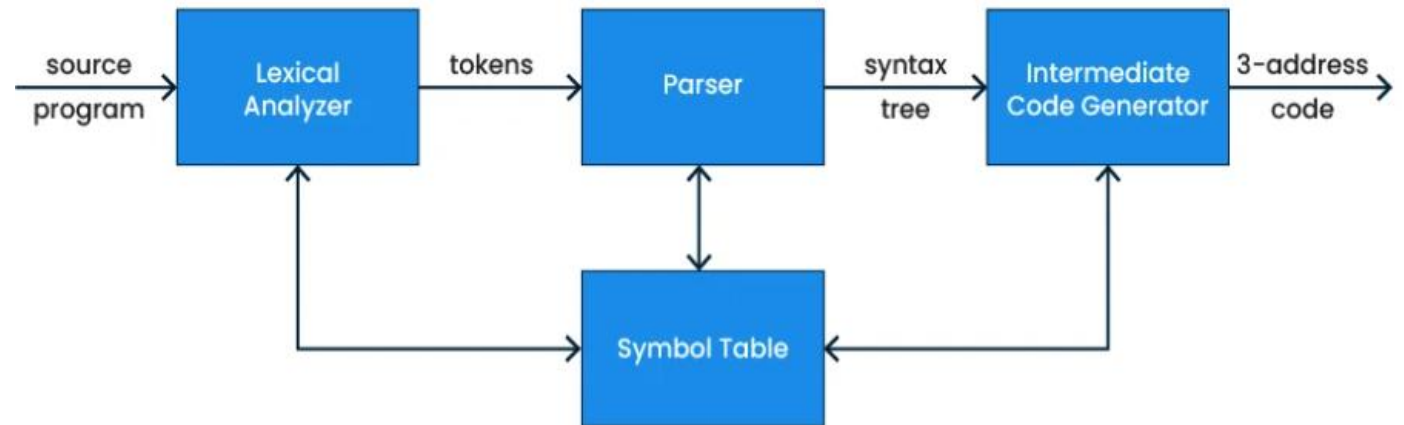
- Intermediate Code Generation is the fourth phase of a compiler.
- In this phase, the compiler converts the source program into an Intermediate Representation (IR), which is an abstract and machine-independent code.
- The intermediate code acts as a bridge between the Front End and Back End of the compiler.
- Since it is independent of hardware, it makes the code easier to improve and allows the same program to be used on different systems.

Main tasks of Intermediate Code Generation:

- Converts human-readable source code into an intermediate form.
- Provides a common representation for code optimization.
- Helps generate machine code for different hardware systems.

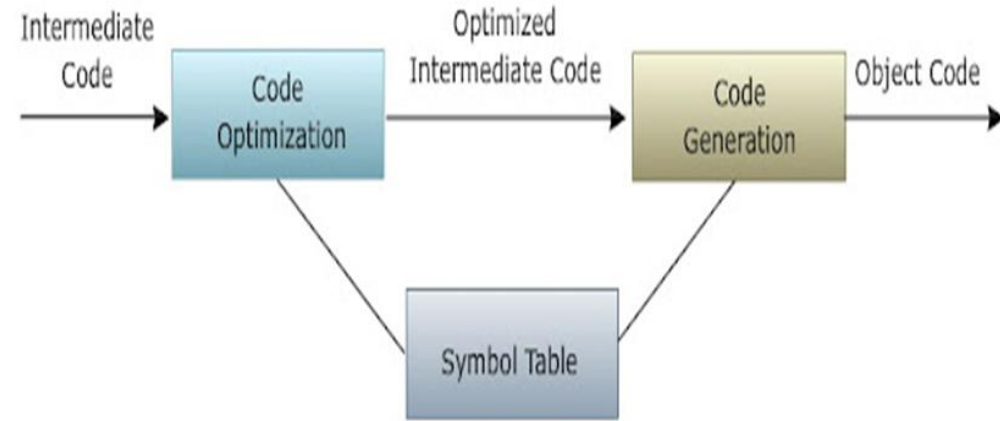
Intermediate Code Formats:

- ✓ Three-Address Code (TAC)
- ✓ Syntax Trees
- ✓ Directed Acyclic Graphs (DAGs)
- ✓ Bytecode



Code Optimization

- **Code Optimization** is the **fifth phase of a compiler**.
- It improves the **Intermediate Code** to make the program more efficient without changing its original functionality.
- The optimized code helps the program:
 - Run faster
 - Use less memory
 - Consume fewer system resources

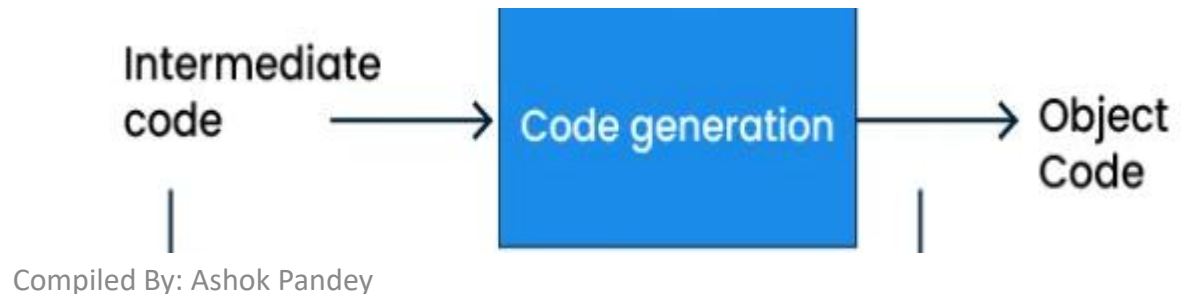


Objectives of Code Optimization:

- **Maintain Correctness:** The optimized code should give the same result as the original code.
- **Improve Performance:** Make the program run faster and use fewer resources like memory.
- **Save Compilation Time:** Optimization should not take too much extra time during compilation.
- **Common Optimization Techniques:** Constant Folding, Constant Propagation, Dead Code Elimination, Loop Optimization

Code Generation

- Code Generation is the final phase of a compiler.
- It converts the optimized Intermediate Code (IR) into machine code or target code that can be executed by a specific computer system.
- During this phase, the compiler selects suitable machine instructions and generates code according to the target hardware.
- The main purpose of code generation is to produce correct and efficient target code that can run on the intended computer architecture.
- The output of this phase is the Machine Code (Target Program).

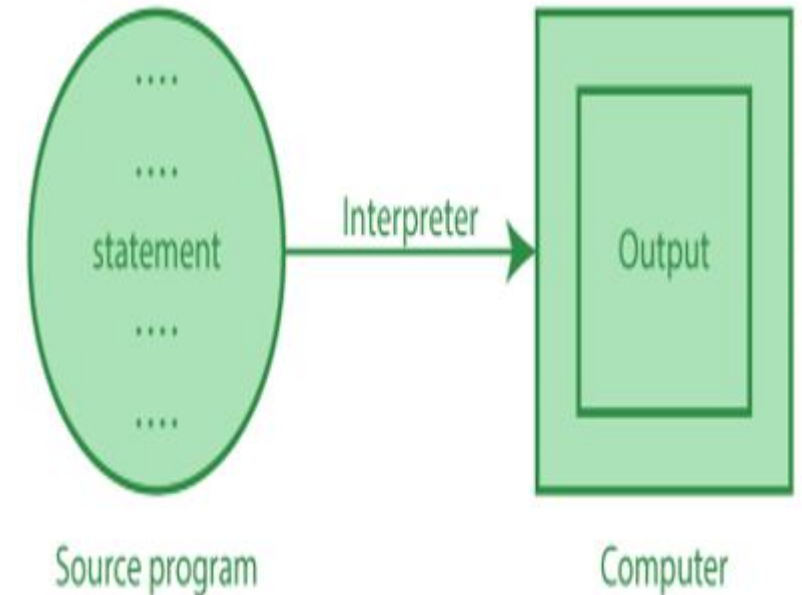


Interpreter

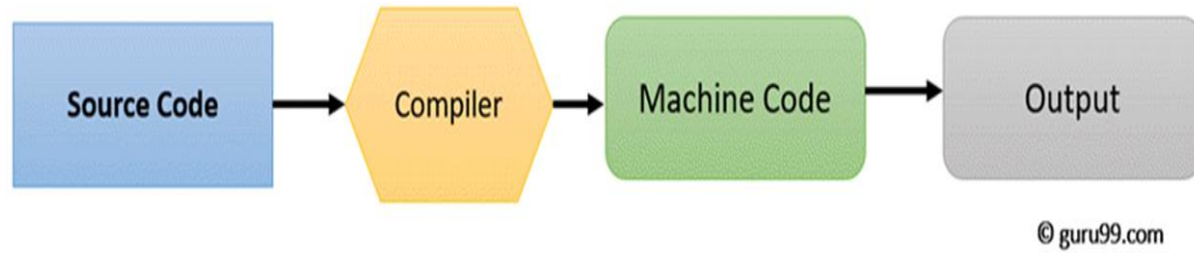
- An Interpreter is a program that executes the source code line by line without creating machine code before execution.
- It translates and runs each statement at runtime.
- Unlike a compiler, an interpreter does not convert the entire program at once.
- It reads one instruction, translates it, and executes it immediately.

Features of Interpreter:

- ✓ Executes the program line by line.
- ✓ Does not generate a separate machine code file before execution.
- ✓ Stops execution immediately when it finds an error.
- ✓ Translates and executes each statement during runtime.



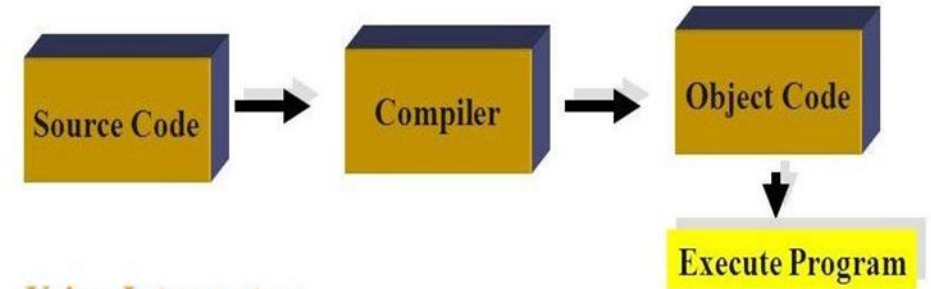
How Compiler Works



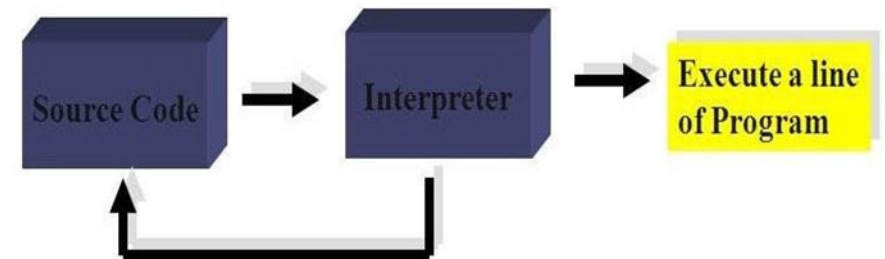
How Interpreter Works



- Using Compiler:



Using Interpreter:



27/27

Compiler vs Interpreter

- A **compiler** and an **interpreter** are both language translators that convert high-level programming code into a format that can be understood by a computer.
- They both work differently when translating and executing code.

Compiler	Interpreter
Translates the entire program at once before execution.	Translates and executes the program line by line.
Generates machine code or an executable file after compilation.	Do not generate a separate machine code file.
Execution is faster because the code is already translated.	Execution is slower because translation happens during runtime.
Displays all errors after checking the complete program.	Stops execution immediately when it finds an error.
Requires more memory to store the generated machine code.	Requires less memory because code is translated when needed.
Program needs to be compiled again after making changes.	Program can be executed directly after making changes.
Example: C, C++	Example: Python, JavaScript

One-Pass Compiler

- A **One-Pass Compiler** is a compiler that processes the source program **only once** from beginning to end.
- It reads the code, analyzes it, and generates the target code in a single pass.
- In this type of compiler, all compilation tasks are performed together without going back to previous parts of the program.

Features of One-Pass Compiler:

- Processes the program **only one time**.
- Compilation is faster because it requires less processing.
- Uses less memory compared to multi-pass compilers.
- Suitable for simple programming languages.

Example :

```
int x = 10;
```

The compiler reads this statement once:

- Reads: `int x = 10;`
- Analyzes: Checks the keyword (`int`), variable name (`x`), and value (`10`).
- Generates: Produces the corresponding target code immediately.

It does not go back and read the statement again.

One-Pass VS Multi-Pass Compiler

Advantages:

- Faster compilation speed.
- Uses less memory.
- Simple design and implementation.

Disadvantages:

- Limited code optimization.
- Cannot easily handle complex programming structures.
- Error checking and recovery are more difficult.

One-Pass Compiler:

- The compiler reads the source code **only one time**.
- It analyzes the code and generates the target code **during the same pass**.
- It is faster but has limited optimization ability.

Source Code → Read & Analyze → Generate Code → Output

Multi-Pass Compiler:

- The compiler reads and processes the source code **multiple times**.
- Each pass performs a different task, such as analysis, optimization, and code generation.
- It takes more time but can produce more efficient code.

Source Code → Analysis → Optimization → Code Generation → Output

Preprocessor

A Preprocessor is a program that runs before the compiler starts. It processes the source code and prepares it for the compilation process.

Functions of Preprocessor:

- File Inclusion:

Adds the contents of external files into the program.

- Macro Expansion:

Replaces macro names with their predefined values before the compiler processes the code.

- Example:

`#include<stdio.h>`

- This instruction tells the preprocessor to insert the contents of the standard input-output header file into the program before compilation begins.
- The preprocessor modifies and prepares the source code before it is given to the compiler.

Macros

- Macros are named pieces of code or constant values that are replaced by the preprocessor before the compilation process starts.
- They are created using the `#define` directive and help reduce repeated code and make programs easier to understand.
- Example:

`#define PI 3.14159`

In this example, every occurrence of PI in the program is replaced with 3.14159 before compilation.

Macros:

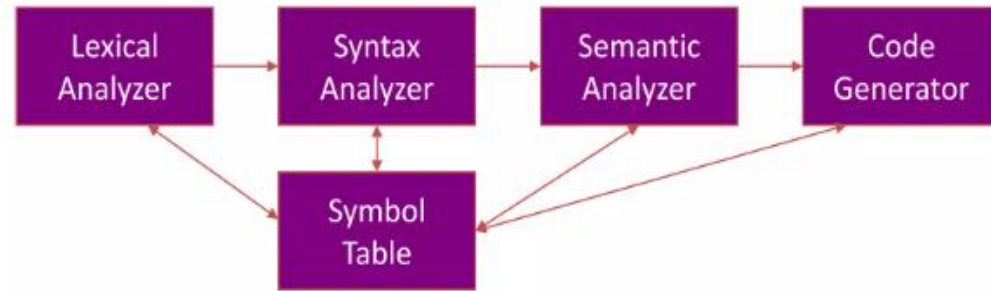
- ✓ Work during the preprocessing stage.
- ✓ Do not require memory allocation like variables.
- ✓ Improve code readability and reusability.
- ✓ Help avoid writing the same value or code multiple times.

Symbol Table

- A Symbol Table is a data structure used by a compiler to store and manage information about identifiers used in a program.
- It keeps details about program elements such as variables, functions, classes, and objects, along with their attributes like data type, scope, and memory location.
- The symbol table helps the compiler find and track identifiers during the entire compilation process.
- It is created during the Lexical Analysis and Syntax Analysis phases and is used by both the Front End and Back End of the compiler.

Symbol Table stores following Information :

- Identifier Name: Name of variables, functions, or other program elements.
- Data Type: Type of data stored (such as integer, float, or character).
- Scope Level: The area of the program where the identifier can be accessed.
- Memory Location: The location where data is stored in memory.
- Function Parameters: Information about function arguments and their types.



Identifier Name	Data Type	Scope	Memory Location	Additional Information
x	int	Local	1000	Variable
sum	function	Global	2000	Function
name	char	Local	1004	Variable
add()	function	Global	3000	Parameters: int a, int b

Symbol Table Structure

Compiled By: Ashok Pandey

Error Handler

- An Error Handler is a component of a compiler that detects, reports, and manages errors during the compilation process.
- It helps identify problems in the source code and provides useful error messages to the programmer.

Functions of Error Handler:

- Detects Errors: Finds errors in the program during compilation.
- Reports Errors: Displays clear and meaningful error messages.
- Error Recovery: Attempts to recover from errors so that the compilation process can continue.

Types of Errors Handled:

- Lexical Errors: Errors caused by invalid characters or incorrect tokens.
 - Syntax Errors: Errors caused by incorrect grammar or program structure.
 - Semantic Errors: Errors related to the meaning of the program, such as type mismatches or undeclared variables.
- The main purpose of an error handler is to help the compiler continue working after finding errors and identify multiple errors in one compilation.

Types of Errors Handled by Error Handler:

1. Lexical Errors

- Lexical errors occur when the compiler finds invalid characters, symbols, or tokens that are not allowed in the programming language.
- These errors are detected during the Lexical Analysis phase.

✓ Example: `int x = 10@`

Here, @ is an invalid character, so the compiler reports a lexical error.

2. Syntax Errors

- Syntax errors occur when the program does not follow the grammar rules of the programming language.
- These errors are detected during the Syntax Analysis phase.

Examples:

✓ Missing semicolon: `int x = 10`

✓ Unmatched brackets: `if (x > 5 { }`

3. Semantic Errors

- Semantic errors occur when the program structure is correct, but the meaning or logic of the program is incorrect.
- These errors are detected during the Semantic Analysis phase.

Examples:

✓ Type mismatch:

```
int x;  
x = "Hello";
```

An integer variable cannot store a string value.

✓ Undeclared variable:

```
y = 10;
```

The variable y is used without being declared.

✓ Invalid operation:

```
int x = 10 / 0;
```

Division by zero is not a valid operation.